

Gestión de la Información en la Web
Curso 2026–2027
Práctica 2 – Formatos CSV y JSON

Fecha de entrega: domingo 11 de octubre

Entrega de la práctica

La entrega de la práctica se realizará a través del Campus Virtual de la asignatura mediante un fichero `pr2.py`. El esqueleto de este fichero se puede descargar del Campus Virtual.

Lenguaje de programación

Python 3.13 o superior.

Calificación

Se medirá la corrección mediante tests de unidad. Además de la corrección, se valorará la **calidad, concisión y claridad del código**, la incorporación de **comentarios** explicativos, su **eficiencia** tanto en tiempo como en memoria y la puntuación obtenida en Pylint.

Declaración de autoría e integridad

Todos los ficheros entregados contendrán una cabecera en la que se indique la asignatura, la práctica, el grupo y los autores. Esta cabecera también contendrá la siguiente declaración de integridad:

Declaramos que esta solución es fruto exclusivamente de nuestro trabajo personal. No hemos sido ayudados por ninguna otra persona o sistema automático ni hemos obtenido la solución de fuentes externas, y tampoco hemos compartido nuestra solución con otras personas de manera directa o indirecta. Declaramos además que no hemos realizado de manera deshonesto ninguna otra actividad que pueda mejorar nuestros resultados ni perjudicar los resultados de los demás.

No se corregirá ningún fichero que no venga acompañado de dicha cabecera.

1. Formato CSV [5pt]

Considerar el archivo `AccidentesBicicletas_2025.csv`. El archivo contiene información sobre los accidentes en los que se han visto involucradas bicicletas en la ciudad de Madrid durante el año 2025. Se puede encontrar más información del conjunto de datos en <https://datos.madrid.es/dataset/300110-0-accidentes-bicicleta/downloads>.

Se pide crear funciones que realicen las siguientes operaciones.

IMPORTANTE: en las funciones que devuelven diccionarios, el orden en el que aparecen las claves a la hora de imprimirlas por pantalla es completamente irrelevante. Las claves de los diccionarios no tienen una noción de orden, lo único importante es que sean diccionarios Python bien contruidos y que las claves tengan asociados los valores correctos.

IMPORTANTE: abre el fichero CSV con el encoding `utf-8-sig` para leer correctamente el BOM («Byte Order Mark») del fichero tal cual te lo descargas.

1. (1pt) `lee_fichero_accidentes(ruta)`

Leer los datos del archivo y devolverlos en una lista de diccionarios, una por cada línea. El resultado con el fichero `AccidentesBicicletas_2025.csv` debería empezar así:

```
1 [{ 'num_expediente': '2025S000088',
2   'fecha': '01/01/2025',
3   'hora': '17:10:00',
4   'localizacion': 'CALL. OCA / CALL. GENERAL RICARDOS',
5   'numero': '2',
6   'cod_distrito': '11',
7   'distrito': 'CARABANCHEL',
8   'tipo_accidente': 'Otro',
9   'estado_meteorológico': 'Despejado',
10  'tipo_vehiculo': 'Bicicleta',
11  'tipo_persona': 'Conductor',
12  'rango_edad': 'De 35 a 39 años',
13  'sexo': 'Hombre',
14  'cod_lesividad': '7',
15  'lesividad': 'Asistencia sanitaria sólo en el lugar del accidente',
16  'coordenada_x_utm': '437920',
17  'coordenada_y_utm': '4471163',
18  'positiva_alcohol': 'N',
19  'positiva_droga': '' },
20 { 'num_expediente': '2025S000097',
21   'fecha': '01/01/2025',
22   'hora': '19:45:00',
23   'localizacion': 'CALL. GENEROSIDAD / CALL. VILLAFUERTE',
24   'numero': '22',
25   'cod_distrito': '17',
26   'distrito': 'VILLAVERDE',
27   'tipo_accidente': 'Caída',
28   'estado_meteorológico': 'Despejado',
29   'tipo_vehiculo': 'Bicicleta',
30   'tipo_persona': 'Conductor',
31   'rango_edad': 'De 30 a 34 años',
32   'sexo': 'Hombre',
33   'cod_lesividad': '7',
34   'lesividad': 'Asistencia sanitaria sólo en el lugar del accidente',
35   'coordenada_x_utm': '441441',
36   'coordenada_y_utm': '4467570',
```

```
37 'positiva_alcohol': 'N',
38 'positiva_droga': ''}
39 ...
40 ]
```

2. (1pt) `accidentes_por_distrito_tipo(datos)`

Obtener un diccionario que por cada distrito (columna `distrito`) y por cada tipo de accidente (columna `tipo_accidente`) indica cuántos accidentes hubo. Esta función acepta como parámetro `datos` la lista de diccionarios obtenida del fichero CSV. El resultado de invocar a esta función con los datos del fichero `AccidentesBicicletas_2025.csv` sería un diccionario como el siguiente:

```
1 {'CARABANCHEL', 'Otro': 5,
2  ('VILLAVERDE', 'Caída'): 14,
3  ('PUENTE DE VALLECAS', 'Alcance'): 6,
4  ('LATINA', 'Atropello a persona'): 4,
5  ('CHAMBERÍ', 'Colisión lateral'): 12,
6  ('CENTRO', 'Caída'): 49,
7  ...
8  }
```

3. (1pt) `dias_mas_accidentes(datos)`

Función que devuelve las fechas del día o días con más accidentes, junto con ese número de accidentes, tomando como entrada una lista de diccionarios como en el primer apartado. Se debe devolver un **conjunto de parejas** (día, número de accidentes). La salida esperada para el fichero `AccidentesBicicletas_2025.csv` sería:

```
1 {'08/06/2025', 13}
```

4. (2pt) `puntos_negros_distrito(datos, distrito, k)`

Función que dado un distrito, genera una **lista** con el *top-k* de localizaciones donde más accidentes se han producido. Esa lista incluirá parejas (*localización*, *número de accidentes*) ordenada de manera descendente por número de accidentes. En caso de empate a número de accidentes, se mostrarán primero las localizaciones con un nombre lexicográficamente mayor, es decir, 'PASEO. PIÑONERO, 4' deberá aparecer antes de 'CTRA. PARQUE DE ATRACCIONES'. Para el fichero 'AccidentesBicicletas_2025.csv' y el distrito 'MONCLOA-ARAVACA', el top-16 debería ser:

```
1 [('PASEO. SENDA DEL REY / CALL. OBISPO TREJO', 6),
2  ('CTRA. RODAJOS, 2', 6),
3  ('CTRA. CIUDAD UNIVERSITARIA, 0', 5),
4  ('PASEO. CASTAÑOS, 0', 4),
5  ('PTA. RODAJOS, 0', 3),
6  ('PASEO. EMBARCADERO, 1', 3),
7  ('CASA DE CAMPO', 3),
8  ('RONDA. TREN / CALL. CARABO', 2),
9  ('PLAZA. MORERAS, 1', 2),
10 ('PASEO. PIÑONERO, 8', 2),
11 ('PASEO. CASTAÑOS / platanos', 2),
12 ('PASEO. CAMOENS Y VALERO, 6 FAROLA 18', 2),
13 ('GTA. SAN VICENTE, 1', 2),
14 ('GTA. PATINES, 0', 2),
15 ('CTRA. ZARZON, 6', 2),
16 ('CTRA. CIUDAD UNIVERSITARIA / CMNO. VALDEZA', 2)]
```

2. Formato JSON [5pt]

Considerar el archivo `300356-2-monumentos-ciudad-madrid-json.json`. El archivo contiene información sobre los monumentos de Madrid, y ha sido obtenido de la página web de datos en abierto de la ciudad de Madrid (<https://datos.madrid.es/dataset/300356-0-monumentos-ciudad-madrid>). En este apartado hay que implementar una serie de funciones para consultar este fichero JSON:

1. (1pt) `leer_monumentos(ruta)`

Esta función acepta la ruta del fichero JSON y devuelve una lista de monumentos, cada uno representado como un diccionario Python.

2. (1pt) `codigos_postales(monumentos)`

Esta función debe aceptar la lista de monumentos y devolverá una **lista** de parejas de tipo `(código_postal, número_de_monumentos)` con el número total de documentos que hay en cada código postal. Las parejas del resultado deben de aparecer en orden descendente por número de monumentos, y en caso de empate se debe conservar el orden en el que aparecen en el fichero JSON. Un ejemplo de su salida sería:

```
1 [(' ', 1471),
2  ('28046', 38),
3  ('28003', 27),
4  ('28032', 21),
5  ('28010', 21),
6  ('28027', 19),
7  ('28002', 14),
8  ('28008', 13),
9  ('28036', 12),
10 ('28015', 12),
11 ('28040', 12),
12 ('28020', 12),
13 ('28004', 10),
14 ('28013', 9),
15 ('28042', 9),
16 ...
17 ]
```

3. (1pt) `busqueda_palabras_clave(monumentos, palabras)`

Esta función recibe una lista de monumentos y una lista de palabras clave, y devuelve un **conjunto de parejas** `(título, distrito)` de aquellos monumentos que contienen las palabras clave en su título o en su descripción (campo `'organization-desc'`). Es necesario que aparezcan **todas las palabras clave entre los dos campos** para considerar que el monumento encaja con la búsqueda, pero **no hay que tener en cuenta las mayúsculas o minúsculas**. Para considerar que una palabra clave aparece en un texto *debe aparecer como subcadena en cualquier parte*, es decir, un texto como «muy próxima al neobarroco, con esculturas y jarrones» contendrá la palabra clave «esculturas» pero también «escultura» y «cultura» porque son subcadenas aunque no aparezcan como palabras completas. El valor de distrito será la URL almacenada en el campo `"@id"` dentro de `address > district`, y si este campo no existe el valor de distrito será la cadena vacía `('')`.

La salida esperada para los monumentos que encajan con las palabras clave `['escultura', 'agua']` sería el siguiente conjunto:

```
1 {('Alfonso XII',
2  'https://datos.madrid.es/egob/kos/Provincia/Madrid/Municipio/Madrid/Distrito/Distrito'),
```

```
3 ('Bravo Murillo',
4  'https://datos.madrid.es/egob/kos/Provincia/Madrid/Municipio/Madrid/Distrito/Chamberi'),
5 ('Dodecaedro-Éter',
6  'https://datos.madrid.es/egob/kos/Provincia/Madrid/Municipio/Madrid/Distrito/Arganzuela'),
7 ('Estanque Descubrimiento',
8  'https://datos.madrid.es/egob/kos/Provincia/Madrid/Municipio/Madrid/Distrito/Salamanca'),
9 ('Felipe IV',
10 'https://datos.madrid.es/egob/kos/Provincia/Madrid/Municipio/Madrid/Distrito/Centro'),
11 ('Francisco de Quevedo y Villegas',
12 'https://datos.madrid.es/egob/kos/Provincia/Madrid/Municipio/Madrid/Distrito/Chamberi'),
13 ('Fuente Poliedros Maclados en el Cubo',
14 'https://datos.madrid.es/egob/kos/Provincia/Madrid/Municipio/Madrid/Distrito/SanBlas'),
15 ('Fuente de Cibeles',
16 'https://datos.madrid.es/egob/kos/Provincia/Madrid/Municipio/Madrid/Distrito/Retiro'),
17 ('Fuente de la Cruz Verde',
18 'https://datos.madrid.es/egob/kos/Provincia/Madrid/Municipio/Madrid/Distrito/Centro'),
19 ...
20 }
```

4. (2pt) `busqueda_distancia(monumentos, direccion, distancia)`

Función que devuelve una **lista de ternas** (título, id, distancia en kilómetros) de los monumentos que se encuentran a menos de la distancia indicada desde la dirección pasada como parámetro. La lista de monumentos deberá estar ordenada de más cercano a lejano, y **en caso de empate a distancia se debe preservar el orden original de los monumentos en el listado original**.

Para conocer la latitud y longitud de una dirección usaremos la biblioteca GeoPy. Aquí tenéis un ejemplo de uso para conocer la latitud y longitud de la dirección donde se encuentra la Facultad de Informática, 'Profesor José García Santesmases 9, Madrid, España':

```
1 >>> from geopy.geocoders import Photon
2
3 >>> geolocator = Photon(user_agent="giw_practica2")
4 >>> location = geolocator.geocode("Profesor José García Santesmases 9, Madrid, España")
5
6 >>> print((location.latitude, location.longitude))
7 (40.4527989, -3.7337564)
```

Por otro lado, para calcular la distancia entre dos puntos dados como (**latitud, longitud**)¹ usaremos la misma biblioteca GeoPy, en este caso el módulo `distance`:

```
1 >>> from geopy import distance
2
3 >>> wellington = (-41.32, 174.81) # Aeropuerto de Wellington
4 >>> salamanca = (40.96, -5.50) # Aeropuerto de Salamanca
5
6 >>> print(distance.distance(wellington, salamanca).km)
7 19959.67926735382
```

IMPORTANTE: existen algunas entradas del fichero JSON que no contienen información de localización. Vuestra función debe ignorar estos monumentos.

La salida esperada de los monumentos a menos de 1 km de la calle "Profesor José García Santesmases 9, Madrid, España" sería la siguiente:

¹Es muy importante usar parejas (latitud, longitud) y no al revés para invocar a la función `distance.distance`: <https://geopy.readthedocs.io/en/stable/#module-geopy.distance>

```
1 [('José Ortega y Gasset', '400073', 0.37591766185928116),  
2  ('Camilo José Cela', '408914', 0.4645580479988918),  
3  ('Alfonso XIII', '400063', 0.6612427126777463),  
4  ('Puerta de Hierro', '400408', 0.9218429100319819),  
5  ('Blas Lázaro e Ibiza', '409134', 0.9949673440389073)]
```